

GA VRPTW UI Application

A modern graphical interface for running genetic algorithm experiments on Vehicle Routing Problem with Time Windows (VRPTW) instances.

Features

-  **Easy Data Selection:** Browse and select multiple data files from the `data/` folder or add custom files
-  **Operator Configuration:** Choose from multiple initialization, crossover, and mutation operators
-  **Flexible Parameters:** Configure population size, generations, mutation rate, and number of runs
-  **Results Visualization:** View convergence curves, fitness distributions, and statistics
-  **Multiple Combinations:** Automatically run experiments with different operator combinations
-  **Background Execution:** Experiments run in background without freezing the UI

Installation

Ensure you have the required dependencies:

```
pip install matplotlib numpy
```

Python 3.7+ with tkinter (usually pre-installed with Python)

Quick Start

Option 1: Using the launcher

```
python run_ui.py
```

Option 2: Direct execution

```
cd vrptw/view  
python app.py
```

Usage Guide

Tab 1: Configuration

1. **Select Data Files**

- Click " Browse Data Folder" to select files from the **data/** directory
- Use "**+** Add Custom File" to add files from other locations
- " Clear" to remove all selections

2. Select Operators

- **Initialization:** Random, Solomon, Cluster First, Savings, or Mixed
- **Crossover:** PMX, Order (OX), Cycle (CX), Edge Assembly, Route-Based
- **Mutation:** Or-Opt, Scramble, 2-Opt, Insert
- **Selection:** Tournament, Roulette, or Truncation

Select at least one option for each type. The app shows the number of combinations.

3. Configure GA Parameters

- **Population Size:** Number of individuals (10-500)
- **Generations:** Number of GA iterations (10-1000)
- **Mutation Rate:** Mutation probability (0.0-1.0)
- **Number of Runs:** Independent runs for each configuration

4. Run Experiment

- Click " Run Experiment" to start
- Progress bar shows completion percentage
- Status updates in real-time

Tab 2: Results

1. View all completed experiments in the list
2. Select an instance to inspect
3. Click " Plot Selected" to visualize:
 - Convergence curves for all runs
 - Fitness distribution (box plot + scatter)
 - Statistical summary

PROF

Available Operators

Initialization Functions

- **Random:** Greedy random insertion
- **Solomon:** Solomon's nearest insertion heuristic
- **Cluster First:** Cluster-first route-second approach
- **Savings:** Savings algorithm (Clarke-Wright)
- **Mixed:** Randomly selects between multiple heuristics

Crossover Operators

- **PMX:** Partially Mapped Crossover
- **Order (OX):** Order Crossover
- **Cycle (CX):** Cycle Crossover

- **Edge Assembly:** VRPTW-specific edge-based crossover
- **Route-Based:** Route-level crossover for VRPTW

Mutation Operators

- **Or-Opt:** Or-Opt move (relocate sequences)
- **Scramble:** Random shuffling of subsequences
- **2-Opt:** 2-Opt swap moves
- **Insert:** Random insertion moves

Selection Methods

- **Tournament:** Tournament selection with k=3
- **Roulette:** Fitness-proportionate selection
- **Truncation:** Select best individuals

File Organization

```
genetic_algorithm/
├── run_ui.py           # Launcher script
├── vrptw/
│   └── view/
│       ├── app.py     # Main GUI application
│       └── main.py    # Original view module
├── data/              # VRPTW benchmark instances
├── ga/                # Genetic algorithm implementation
├── operators/         # Crossover and mutation operators
└── results/           # Experiment results (auto-created)
```

Results Storage

Results are automatically saved in the **results/** folder:

```
results/
├── C101/
│   ├── run_001.json
│   ├── run_002.json
│   └── summary.json
├── R101/
│   ├── run_001.json
│   └── ...
```

Each result file contains:

- Best fitness per run
- Convergence data (best and average fitness per generation)

- Operator statistics
- Runtime information

Tips & Tricks

Performance

- Start with fewer generations (50-100) for quick tests
- Use smaller population sizes (20-30) for faster iterations
- Fewer runs (1-2) for testing configurations

Experimentation

- Start with one operator combination to validate the setup
- Gradually increase complexity (more operator options)
- Run multiple instances to compare algorithm robustness

Analysis

- Check "Plot Selected" after each experiment run
- Compare different operator combinations via the results tab
- Look at convergence curves to identify operator performance

Troubleshooting

"Instance file not found" error

- Ensure data files are in the **data/** folder
- Use "Add Custom File" if files are elsewhere

Plots not showing

- Ensure matplotlib and numpy are installed
- Check that result files exist in the **results/** folder

UI appears frozen during run

- This is normal - the progress bar updates as experiments complete
- Background threading prevents UI freezing

Example Workflows

Quick Test

1. Select 1-2 small instances (e.g., C101, C102)
2. Choose 1-2 operators from each type
3. Set Population=20, Generations=50, Runs=1
4. Run and view results in minutes

Comprehensive Comparison

1. Select all instances
2. Choose multiple operators for each type
3. Set Population=50, Generations=200, Runs=3
4. Run overnight or let it run while you work

Specific Operator Comparison

1. Select one instance
2. Try all combinations of crossover and mutation
3. Keep initialization constant
4. Compare results in the Results tab

Advanced Usage

To modify available operators, edit [vrptw/view/app.py](#):

```
INITIALIZERS = {  
    "Custom": custom_function,  
    ...  
}  
  
CROSsoVERS = {  
    "Custom": custom_crossover,  
    ...  
}
```

License

As part of the Genetic Algorithm for VRPTW project.